Hardware and Software for Big Data mod. A

Academic Year 2025–2026

A Distributed Sentiment Analysis System for Large-Scale Twitter(X) Data Using Apache Spark and Kafka

Project Report

Authors

Isabella Di Lorenzi D03000224

Professor

Prof. Giancarlo Sperlì

2026

Abstract

Sentiment analysis on Twitter data attracts a lot of attention due to the platform's immediacy and ease of communication, which encourages users to freely express opinions and emotions on a wide range of topics. As a result, Twitter represents a continuously growing source of textual data that can be exploited to identify sentiment tendencies and public opinion trends. However, the massive volume of tweets generated daily makes manual analysis infeasible, requiring automated and scalable sentiment analysis techniques.

This work proposes a distributed sentiment analysis system based on Apache Spark and Apache Kafka, designed to handle large-scale Twitter data efficiently, combining stream processing and batch analytics to efficiently handle textual data at scale. Kafka is used as a streaming platform for data ingestion, while Spark is used for text preprocessing, feature extraction, and model training.

Logistic Regression from Spark MLlib is evaluated as a machine learning algorithm to classify tweets into multiple sentiment categories, while the overall model is assessed using accuracy, precision, and recall, demonstrating the effectiveness and scalability of the proposed solution.

Contents

Chapter 1

Challenge Description

1.1 Problem Definition: Sentiment Analysis

Sentiment analysis is the process of using natural language processing (NLP), text analysis, and computational linguistics to identify, extract, and quantify subjective information from text.

A sentiment is a generally binary opposition in opinions and expresses feelings in the form of emotions, attitudes, opinions, and related constructs. It can represent many perspectives, such as *like* or *dislike*, *good* or *bad*, *for* or *against*, among others.

By using machine learning methods and *NLP*, it's possible to automatically extract personal opinions from a document and attempt to classify them according to their sentiment polarity, such as *positive*, *neutral* or *negative*. This makes sentiment analysis an important tool in determining the overall opinion of a defined objective, such as product reviews, customer feedback and social media discussions.

Despite being widely adopted, sentiment analysis is challenging and far from fully solved, since most languages are highly complex due to factors such as objectivity, subjectivity, negation, vocabulary, grammar, and contextual meaning.

Today, sentiment analysis is prevalent in many real-world scenarios to analyze different situations, such as, for example, how Twitter users' attitudes have changed toward an elected president since an election, determining whether a client's email expresses satisfaction or dissatisfaction, or identifying whether a product review is positive or negative.

In general, sentiment analysis offers a very powerful framework for understanding what textual data means from a human perspective, even if its use in large-scale applications and continuously generated data requires scalable

and efficient approaches to overcome its intrinsic challenges.

1.2 Motivation for Choosing Sentiment Analysis

The choice of sentiment analysis as the focus of this project is motivated by an interest in textual data analysis and natural language processing. Text data allows a direct observation of how computational models interpret human language, making sentiment analysis particularly compelling, considering that its results can be easily understood and validated.

Social media platforms are something that nowadays all of us use. Most of us generate and have generated a vast amount of opinionated data over time, and understanding how these data can be processed to automatically extract meaningful information out of it, such as sentiment or emotional tone, represents a very motivating aspect of sentiment analysis.

From a Big Data perspective, sentiment analysis on Twitter data also presents some interesting challenges to explore distributed frameworks and to understand how data processing and machine learning techniques can be applied to large-scale and unstructured datasets.

1.3 Main Challenges in Applying Sentiment Analysis to Twitter Data

Social network platforms have the benefit of making the entire world a small village, where users can share their views, feelings, experiences, and advice. Since many people use social networks daily, a huge amount of data has been created and stored in order to get direct information about and from users. This large-scale and continuously growing volume of content generated by users introduces significant and unique challenges for sentiment analysis.

Specifically, Twitter data is highly unstructured and informal, often containing abbreviations, slang, hashtags, mentions, emojis, and URLs, which increase noise and make it more complicated to interpret textual data. Sarcasm and irony, for example, are easily misclassified in current models, underlying a persistent limitation of sentiment classifiers trained on literal meanings without pragmatic context. In addition, tweets are limited in length, and sometimes this makes it harder to capture sufficient contextual information and detect all those things that fall into the category of complex linguistic phenomena.

The massive scale and unstructured nature of Twitter data exceed the capabilities of traditional data processing systems, which are usually designed for static and structured datasets. As a result, scalable and distributed solutions are necessary to process, analyze, and classify sentiment in Twitter data streams.

Beyond linguistic and scalability issues, Twitter data presents challenges related to data quality and representation: sentiment class distribution is often imbalanced, causing bias classification models toward dominant classes if normalization or balancing techniques are not used in the right way. Sentiment labels can be noisy or subjective, as interpretation of opinions can vary depending on the context. Luckily, the dataset was balanced enough and I didn't face this specific issue in this work.

Chapter 2

Proposed Methodology and Architecture

2.1 Dataset Description

For conducting the analysis, I used a sentiment dataset obtained through Kaggle containing a large collection of tweets with manually or semi-automatically annotated sentiment-related labels.
It's stated that the dataset contains roughly one million records, and each one includes three columns:

- *text*, containing the textual content of a tweet;
- *language*, referring to the language in which it is written;
- *label*, containing the sentiment label associated with the tweet.

Once loaded, the dataset was found to contain 937.854 rows, with all columns represented as string data types.

```
Dataset loaded successfully!
Total rows: 937854
root
 |-- Text: string (nullable = true)
 |-- Language: string (nullable = true)
 |-- Label: string (nullable = true)
```

Figure 2.1: Raw dataset specifications

The text column, as expected from social media data, shows that the text is unstructured and noisy, containing hashtags, user mentions, URLs,

emojis, abbreviations and overall informal language, needing adequate data pre-processing before using it for modeling purposes.

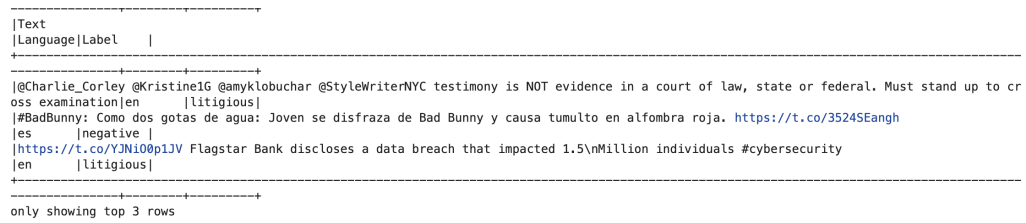


Figure 2.2: Raw text column

The language column presents a multilingual dataset, although the majority of tweets are in English.

```
=== Language Distribution ===
+--------+------+
|Language| count|
+--------+------+
|      en|871310|
|      fr| 13091|
|      es| 11333|
|      pt| 10336|
|      ja|  8414|
|      in|  3000|
|      tl|  2816|
|     und|  2702|
|      de|  2055|
|      tr|  1371|
+--------+------+
only showing top 10 rows
```

Figure 2.3: Distribution of languages in the original dataset

Sentiment labels are provided in a multiclass format, including four categories:

- *positive*, expressing favourable opinions or emotions;
- *negative*, expressing criticism or dissatisfaction;
- *uncertainty*, containing ambiguous or doubtful statements;
- *litigious*, related to disputes or litigation language.

The distribution of labels is fairly balanced and no class dominates the dataset; as a result, the risk of class bias is reduced, and no additional balancing adjustments are needed.

```
=== Sentiment Distribution ===
+-----------+------+
|      Label| count|
+-----------+------+
|   positive|264539|
|   negative|262208|
|uncertainty|206940|
|  litigious|204144|
+-----------+------+
```

Figure 2.4: Distribution of labels in the dataset

Overall, due to its size, unstructured textual content and class diversity, all common characteristics of social media data, I had to use multiple data processing techniques before applying any machine learning model.

2.2 Overall System Architecture

The system follows a distributed data processing and machine learning architecture built on Apache Spark, designed to perform sentiment analysis on large-scale Twitter data. Considering that I am dealing with a large amount of data, it is necessary to use Kafka for queueing it: Kafka has horizontal scalability, enabling to handle large streaming data, moreover, it employs a replication strategy to ensure fault tolerance and data reliability. Once the data are securely queued in Kafka, Spark comes in to analyze the sentiments of these data in a distributed manner. Spark leverages a Master-Slave architecture, allowing for the distributed processing of data and facilitating the efficient analysis of large datasets by parallelizing the workload across multiple workers.

Each component of the pipeline is designed to operate independently and the architecture of the system is organised as a sequence of modular stages, each responsible for a specific task in the workflow. In particular:

- In the *data ingestion layer*, Twitter data is collected and published as a continuous stream, with each tweet treated as an individual data record transmitted in real time. Kafka functions as the message broker, organizing incoming tweets into topics. This structure allows producers and

consumers to operate independently, ensuring system robustness under high data velocity and efficient scalability as data volume increases.

- The *stream processing layer* is implemented using Spark. Spark subscribes to the Kafka topics and continuously reads tweet data in micro-batches; once ingested, the raw tweet data are transformed into structured data frames. Here, applying cleaning, tokenization, and feature extraction, helps to prepare the data for sentiment classification.

- The *sentiment analysis layer* is where I applied a machine learning model trained on historical labeled tweet data, to deduce sentiment labels for incoming tweets in real time.

- Finally, the *output layer* manages the storage and visualization of the sentiment analysis results, making the classified tweets and their sentiment labels available for further analysis, logging, or visualization.

Overall, this layered architecture ensures a smooth flow from ingestion to analysis, allowing the system to reliably process and deliver real-time sentiment insights at scale.

2.3 Data Preprocessing

As observed in Section 2.1, the data extracted from Twitter must be processed.

I observed that the raw dataset contains hashtag, website link, white spaces, emoji, etc., which need to be discarded before processing it, in order for the sentiment produced to be precise. That's why, I proceeded to:

- remove missing values (missing sentiment labels and missing language information), to ensure data consistency during training and evaluation;

- convert all text to lowercase, to prevent duplicated features caused by case differences (e.g., "Hello" and "hello");

- remove all non letters, so I got rid of all punctuation, numbers and special characters;

- remove all links, getting a URLs free clean dataset.

```
Cleaned rows: 937831
+--------------------------------------------------------------+--------------------------------------------------------------+---------+
|                                                          Text|                                                    clean_text|    Label|
+--------------------------------------------------------------+--------------------------------------------------------------+---------+
|@Charlie_Corley @Kristine1G @amyklobuchar @StyleWriterNYC...|charliecorley kristineg amyklobuchar stylewriternyc testi...|litigious|
|#BadBunny: Como dos gotas de agua: Joven se disfraza de B...|badbunny como dos gotas de agua joven se disfraza de bad ...| negative|
|https://t.co/YJNiO0p1JV Flagstar Bank discloses a data br...| flagstar bank discloses a data breach that impacted \nmi...|litigious|
|Rwanda is set to host the headquarters of United Nations ...|rwanda is set to host the headquarters of united nations ...| positive|
|OOPS. I typed her name incorrectly (today's brave witness...|oops i typed her name incorrectly todays brave witness ca...|litigious|
+--------------------------------------------------------------+--------------------------------------------------------------+---------+
only showing top 5 rows
```

Figure 2.5: Table containing raw text, clean text and their labels

Following this, considering that the majority of the dataset has English written content, I restrict the dataset to English tweets only.

```
English tweets: 871310
+--------------------------------------------------------------------------------------------------------------+---------+
|                                                                                                    clean_text|    Label|
+--------------------------------------------------------------------------------------------------------------+---------+
|charliecorley kristineg amyklobuchar stylewriternyc testimony is not evidence in a court of law s...|litigious|
|httpstcoyjniopjv flagstar bank discloses a data breach that impacted \nmillion individuals cybers...|litigious|
|rwanda is set to host the headquarters of united nations development programmes undp new innovati...| positive|
+--------------------------------------------------------------------------------------------------------------+---------+
only showing top 3 rows
```

Figure 2.6: Dataset filtered to English-language tweets

Though, after this step, language is not a relevant data anymore and to prepare the dataset for classification I proceeded to keep only the text and sentiment label.

Moreover, Spark ML requires the target variable to be numeric, so I converted sentiment labels into numerical indices.

```
Label Encoding:
Positive -> 0.0
Negative -> 1.0
Uncertainty -> 2.0
Litigious -> 3.0

+-----------+-----------+
|      Label|label_index|
+-----------+-----------+
|   negative|        1.0|
|uncertainty|        2.0|
|   positive|        0.0|
|  litigious|        3.0|
+-----------+-----------+
```

Figure 2.7: Label encoding

Lastly, I applied:

- Tokenization, splitting each tweet into tokens, to enable textual analysis;
- Stop Word Removal, removing common stop words such as "the", "for" and "a", which carry little semantic meaning.

These preprocessing operations are implemented using Spark SQL functions and Spark ML feature transformers.

2.4 Feature Extraction

Feature extraction is a technique that reduces the complexity of data, while retaining as much task-relevant information as possible. During the extraction process, unstructured data is converted into a more usable format, transforming raw text into numerical representations suitable for machine learning algorithms.

Among different text representation techniques, I chose to use the *TF-IDF* (Term Frequency–Inverse Document Frequency) due to its simplicity, interpretability and effectiveness in capturing important words in large text.

Essentially, TF-IDF evaluates the importance of any keyword in the context on the basis of how frequently it appears in a specific document relative to how frequently it appears across the entire corpus. This means that words that occur often in a particular document but rarely in the rest of the corpus are considered more informative and receive higher weights, while words that appear frequently across many documents (like "the" or "and") are assigned lower weights because they carry little value. This property makes this technique particularly effective for identifying key terms that reflect sentiment in textual data such as tweets.

The first step, though, was splitting the dataset into training and testing sets. I did so in order to prevent overfitting, ensuring the model learns general patterns rather than memorizing the training data, and to get an unbiased evaluation of its performance on new, unseen data.

In particular, I used the training set (80% of the dataset) to allow the model to learn patterns and relations in the data and the testing set (20% of the dataset) to evaluate the model on, ensuring that the model's performance reflects its ability to classify new tweets.

```
Training samples: 697501
Testing samples: 173809
```

Figure 2.8: Dataset split into training (80%) and testing (20%) sets

I set a random seed too, so that the split is deterministic, meaning that the same split can be reproduced for consistent results in experiments.

After splitting the dataset, the next step was to transform the preprocessed and tokenized tweets into numerical feature vectors using TF-IDF. I implemented it in Spark using:

- *CountVectorizer*, that converted the tokenized words into raw term frequency vectors, each containing the counts of each word in the vocabulary for that tweet. I applied a minimum document frequency (*minDF=2*) to ignore words that appear only once, reducing noise from extremely rare terms;
- *IDF*, that scaled the raw term frequency vectors, down-weighting common words that appear in many tweets and up-weighting words that are distinctive to a particular tweet

This produced the final TF-IDF feature vectors, stored in the features column, which represent each tweet numerically while highlighting sentiment-relevant words; these vectors serve as inputs to the Logistic Regression model, allowing it to learn from the most informative words in the dataset while ignoring common, uninformative terms.

2.5 Machine Learning Model

The next step was to train a machine learning model to classify tweets based on their sentiment. For this purpose, I chose Logistic Regression, a classification algorithm that aims to measure and explain the relationship between the instance class and the extracted features from the input, widely used both for binary classification and multiclass one, as in this case. I initialized the Logistic Regression classifier, configuring the model with a maximum of 20 iterations and a regularization parameter of 0.01 to prevent overfitting.

To streamline pre-processing and model training, I implemented a Spark ML Pipeline that combines all stages into a single workflow. The pipeline consists of the steps previously described:

1. *Tokenization*, Section 2.3;
2. *Stop Word Removal*, Section 2.3;
3. *Count Vectorization*, Section 2.4;
4. *TF-IDF Weighting*, Section 2.4;
5. *Logistic Regression Classifier*, Section 2.5.

Once trained, I set the pipeline to generate predictions on the testing set, including the cleaned tweet text, the true sentiment label, the predicted label, and the probability distribution across all classes.

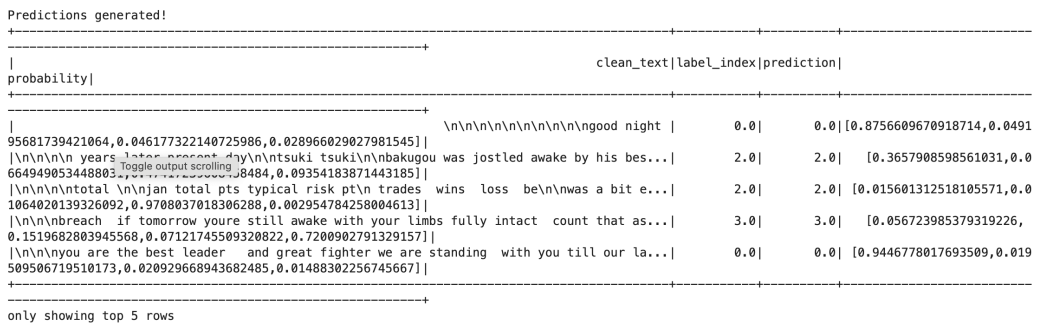


Figure 2.9: Predictions generated by the Logistic Regression model

I ended the section by extracting each trained stage of the pipeline, in order to reuse it for real-time streaming data, ensuring consistent preprocessing and classification of new tweets without retraining the entire model.

2.6 Streaming Data Ingestion with Kafka

Apache Kafka is employed in the system as a distributed messaging platform to enable scalable and fault-tolerant ingestion of tweet data for real-time sentiment classification. Unlike a batch-only setup, where all data must be available before processing begins, Kafka decouples data producers from consumers, allowing tweets to be published and classified continuously as they arrive.

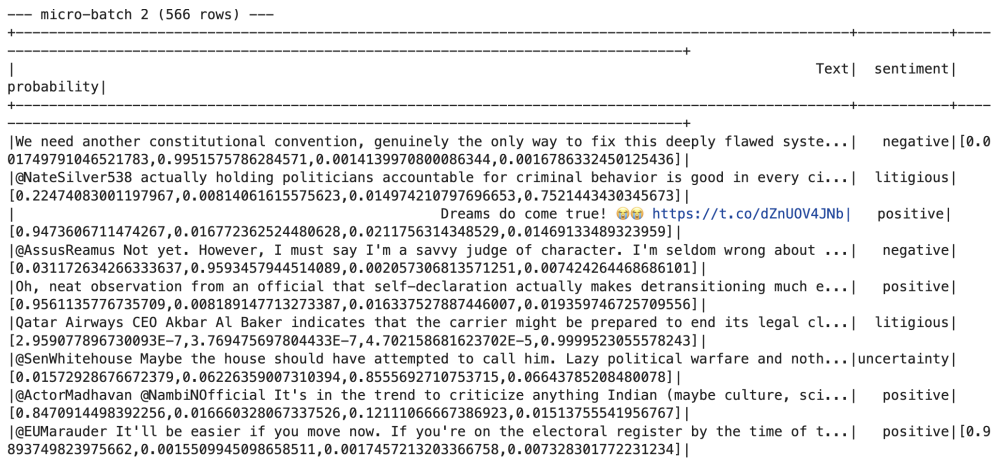
In this architecture, a dedicated Kafka producer publishes raw, unlabeled tweet text to a Kafka topic (tweets-input), simulating a continuous feed of incoming social media content. Spark Structured Streaming subscribes to this

topic and consumes new messages in micro-batches via *spark.readStream.format ("kafka")*, reading directly from the broker rather than from a static file or a synthetic timing source.

Each micro-batch is processed using Spark's *foreachBatch* mechanism, which applies the same fitted *Pipeline* used during offline training and evaluation (tokenization, stopword removal, TF-IDF vectorization, and Logistic Regression classification, as described in Sections 2.3–2.5) directly to the incoming, previously-unseen tweets. This guarantees that streaming predictions are generated using an identical feature representation and decision boundary to the one validated during batch evaluation, with no discrepancy between offline and online inference. Numeric class predictions are then mapped back to their original sentiment labels ("positive", "negative", "uncertainty", "litigious") using an IndexToString transformer fitted on the same label encoding used during training.

To ensure fault tolerance, the streaming query is configured with a checkpoint location, allowing Spark to persist offset and state information and recover cleanly in the event of a restart. For demonstration purposes, the streaming query is run for a bounded duration; in a production setting, this bound can be removed to allow continuous, indefinite processing of the incoming stream.

This design validates that the system performs genuine, end-to-end real-time inference: incoming tweets are unseen, unlabeled text drawn from live traffic on the topic, and the varying class probabilities produced across different tweets confirm that the model is actively discriminating between sentiment categories rather than returning a constant or precomputed output. This distinguishes the pipeline from an offline batch classifier that is merely replayed through Kafka, and better reflects how such a system would operate in a genuine production environment ingesting live social media data.



```
--- micro-batch 2 (566 rows) ---
+--------------------------------------------------------------------------------------------------------+-----------+----
----------------------------------------------------------------------------+
|                                                                                                    Text|  sentiment|
probability|
+--------------------------------------------------------------------------------------------------------+-----------+----
----------------------------------------------------------------------------+
|We need another constitutional convention, genuinely the only way to fix this deeply flawed syste...|   negative|[0.0
01749791046521783,0.9951575786284571,0.0014139970800086344,0.0016786332450125436]|
|@NateSilver538 actually holding politicians accountable for criminal behavior is good in every ci...|  litigious|
[0.22474083001197967,0.00814061615575623,0.014974210797696653,0.7521443430345673]|
|                                                Dreams do come true! 😎😎 https://t.co/dZnUOV4JNb|   positive|
[0.9473606711474267,0.016772362524480628,0.0211756314348529,0.01469133489323959]|
|@AssusReamus Not yet. However, I must say I'm a savvy judge of character. I'm seldom wrong about ...|   negative|
[0.031172634266333637,0.9593457944514089,0.002057306813571251,0.007424264468686101]|
|Oh, neat observation from an official that self-declaration actually makes detransitioning much e...|   positive|
[0.9561135776735709,0.008189147713273387,0.016337527887446007,0.019359746725709556]|
|Qatar Airways CEO Akbar Al Baker indicates that the carrier might be prepared to end its legal cl...|  litigious|
[2.959077896730093E-7,3.769475697804433E-7,4.702158681623702E-5,0.9999523055578243]|
|@SenWhitehouse Maybe the house should have attempted to call him. Lazy political warfare and noth...|uncertainty|
[0.01572928676672379,0.06226359007310394,0.8555692710753715,0.06643785208480078]|
|@ActorMadhavan @NambiNOfficial It's in the trend to criticize anything Indian (maybe culture, sci...|   positive|
[0.8470914498392256,0.016660328067337526,0.12111066667386923,0.01513755541956767]|
|@EUMarauder It'll be easier if you move now. If you're on the electoral register by the time of t...|   positive|[0.9
893749823975662,0.0015509945098658511,0.0017457213203366758,0.007328301772231234]|
```

Figure 2.10: Sample of real-time sentiment predictions generated by Spark Structured Streaming on tweets consumed from the Kafka tweets-input topic.

Chapter 3

Experimental Results

3.1 Experimental Setup

The experimental evaluation of the system was conducted using Apache Spark with PySpark as the development framework. All experiments were performed in a local Spark environment configured to simulate distributed processing, using Python for data pre-processing, model training, and streaming analysis. Apache Kafka was deployed locally as a single-node broker to handle message ingestion for the streaming component.

The sentiment classification model was trained offline using the training dataset and subsequently applied to both test data and streaming tweets. For streaming experiments, unlabeled tweets were published to a Kafka topic (*tweets-input*) and consumed by Spark Structured Streaming in micro-batches, allowing the system to perform real-time sentiment classification on previously unseen data and evaluate performance under continuous data flow.

3.2 Evaluation Metrics

I decided to evaluate the performance of the sentiment analysis model using classification metrics used in natural language processing tasks, in order to get a comprehensive assessment for the model's predictive capabilities.
Specifically, I decided to apply the following:

1. *Accuracy*, the proportion of all classifications that are correct, whether positive or negative, indicating how often a ML model is correct overall;

2. *Precision*, the proportion of all the model's positive classifications that are actually positive, showing how often an ML model is correct when

predicting the target class;

3. *Recall* (also called the true positive rate (TPR)) the proportion of all actual positives that are correctly classified as positives, indicating whether an ML model is able to identify all instances of the target class.

The Logistic Regression classifier achieved a test accuracy, weighted precision and weighted recall of 95.6%, indicating that the TF-IDF feature representation effectively captures sentiment-related information in the text data.

```
================================================
MODEL EVALUATION RESULTS
================================================
Accuracy:          0.9557 (95.57%)
Weighted Precision: 0.9558 (95.58%)
Weighted Recall:   0.9557 (95.57%)
================================================
```

Figure 3.1: Results for accuracy, precision and recall metrics

Misclassifications are relatively limited and occur mainly between neighboring classes, suggesting that the model captures meaningful sentiment patterns.

Moreover, I added a confusion matrix, a table used to measure how well a classification model performs, comparing the predictions made by the model with the actual results, indicating where the model was right or wrong.

```
================================================
CONFUSION MATRIX
================================================
Label            0.0       1.0       2.0       3.0
------------------------------------------------
0.0            47580       964       781       351
1.0              994     46337       859       543
2.0              682       684     37949       217
3.0              496       704       418     34250
================================================
```

Figure 3.2: Confusion matrix

It shows that all classes have roughly similar number of instances, which means that the weighted metrics are not skewed by class imbalance.

3.3 Results and Discussion

The experimental results demonstrate that the system is effective in performing sentiment analysis on both static and streaming Twitter data, achieving very satisfactory accuracy on the test dataset, that indicates how the preprocessing pipeline and feature extraction methods were effective in capturing meaningful patterns in tweet text.

Analysis of precision and recall shows that the model performed consistently across sentiment classes, although some variations were observed, and that can be attributed to different linguistic cues associated with diverse sentiment.

The streaming experiments validated the efficiency of the system, having the incoming tweets processed from Kafka in near real time, ensuring reliable data ingestion and fault tolerance throughout the streaming process.

Future improvements could, for example, include the use of advanced deep learning models or contextual word embeddings to better capture semantic nuances. Overall, the experimental results confirm that the proposed architecture is suitable for scalable, real-time sentiment analysis of streaming Twitter data.